

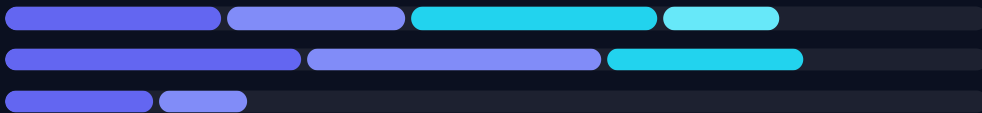


PRODUCT & TECHNICAL SPECIFICATION

ADM

Advance Download Manager

A fast, modern, segmented download manager for Linux,
written in Go, with a first-class batch download workflow.



Version 0.1 draft · **Date** 30 September 2026

Stack Go · Wails v2 · Svelte · TypeScript · Tailwind CSS · SQLite

Platform Linux (X11 / Wayland)

Contents

1	Vision & goals	8	Organization & automation
2	Technology stack	9	UI / UX design
3	Architecture	10	Data model
4	Download engine	11	Non-functional requirements
5	Batch downloads	12	Roadmap
6	Queues, scheduling & limits	13	Risks & open questions
7	Capture & integration		

1 Vision & goals

Linux has capable download tools ([aria2](#) , [wget](#) , [curl](#)), but none pairs a strong engine with a polished desktop experience. ADM aims to be the Linux equivalent of IDM/FDM: it should saturate your connection, survive anything, and be pleasant to use.

Fast

Dynamic multi-connection segmentation keeps every connection busy until the last byte.

Resilient

Pause and resume survive crashes, reboots, network drops and expired links.

Batch-native

Hundreds of files handled as one unit: import, filter, rename, track, and retry only the failures.

Modern

A clean, animated, keyboard-friendly UI with live per-segment visualization and dark/light themes.

Non-goals for v1

- Windows/macOS builds. The architecture allows them later, but they are not tested in v1.
- BitTorrent, HLS/DASH video and site-specific extractors. These are planned for v2+.
- Cloud sync or accounts.

2 Technology stack

LAYER	CHOICE	WHY
Language (core)	Go 1.23+	Goroutines fit concurrent segment workers naturally. The app ships as a single static binary with a strong net/http stack.
Desktop shell	Wails v2 (WebKitGTK 4.1)	Go backend with a web frontend gives full design freedom and native windowing. It builds to one binary of about 10–15 MB.
Frontend	Svelte 5 + TypeScript + Vite	Tiny runtime and fine-grained reactivity. This matters when you re-render dozens of progress bars many times per second.
Styling	Tailwind CSS v4 + design tokens	Consistent spacing, color and theme switching. Easy dark mode.
Icons	Lucide	A clean, consistent stroke icon set.
Persistence	SQLite via modernnc.org/sqlite	Pure Go (no cgo), transactional, and survives crashes.
Rate limiting	golang.org/x/time/rate	Token bucket at global, per-batch and per-job levels.
IPC	Unix socket + small JSON API	Used by the CLI companion and the browser extension's native host.
Packaging	ApplImage, .deb, Flatpak (later)	Covers most distributions.

Why not GTK4/libadwaita or Fyne? libadwaita looks native on GNOME but makes custom visuals (per-segment bars, sparklines, animated cards) hard. Fyne is hard to make feel premium. Wails keeps the engine in Go and gives the UI full web-grade flexibility.

Build requirements

```
sudo apt install build-essential pkg-config libgtk-3-dev libwebkit2gtk-4.1-dev
go install github.com/wailsapp/wails/v2/cmd/wails@latest
wails build -tags webkit2_41
```

3 Architecture

The engine is a standalone Go package with **no UI dependencies**. The desktop app, the CLI and the browser bridge are thin clients of the same core service. Communication is event-driven: the engine publishes state and progress events, and the UI subscribes to them.

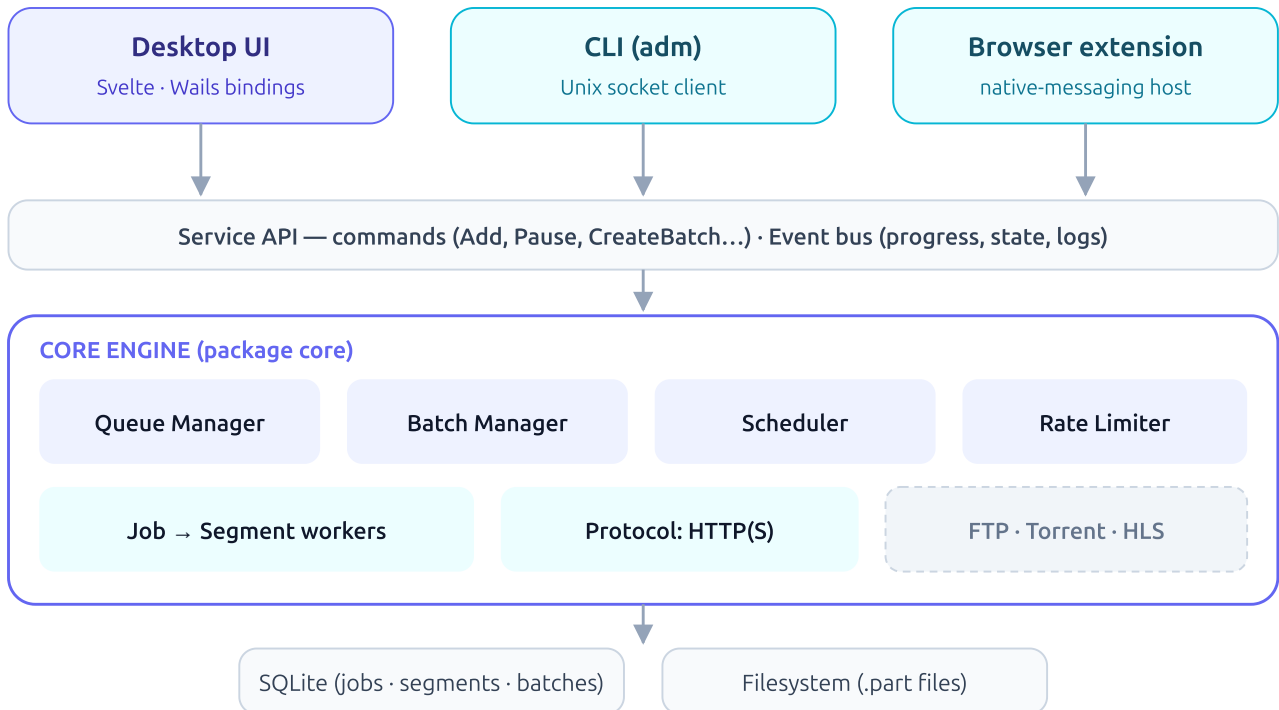


Figure 1. Layered architecture. Dashed boxes are post-v1 protocol handlers.

Repository layout

```
adm/
├─ main.go, app.go           // Wails entry + bindings exposed to the frontend
├─ internal/core/            // engine: manager, job, segment, batch, limiter
├─ internal/store/           // SQLite persistence
├─ internal/batch/           // URL pattern expansion, list parsing, rename templates
├─ internal/probe/           // HEAD/range probing, filename detection
├─ cmd/adm/                  // CLI companion (later)
└─ frontend/                 // Svelte + Tailwind UI
```

4 Download engine

4.1 Job lifecycle

queued → probing → downloading ⇌ paused → verifying → completed, plus failed (retryable) and canceled. Every transition is persisted before it is announced, so the on-disk state is never behind the UI.

4.2 Probing

- Send HEAD. If the server rejects it or gives no length, send GET with Range: bytes=0-0 and read Content-Range.
- Record the size, Accept-Ranges, ETag, Last-Modified, content type and the final URL after redirects.
- Filename priority: Content-Disposition (including RFC 5987 filename*), then the final URL path, then a name generated from the MIME type.

4.3 Dynamic segmentation

A file is split into N initial ranges (default 8, maximum 32). When a worker finishes its range, it **steals work**: it takes the active segment with the most bytes remaining and splits it in half. The original worker keeps the first half and the idle worker takes the second. Splitting stops below a minimum size (1 MiB). This avoids the "one slow tail segment" problem of static splitting.

```
type Segment struct {
    ID          int
    Start, End  int64    // inclusive byte range; End may shrink when stolen from
    Done        int64    // bytes written from Start
}
// Workers write with os.File.WriteAt into one preallocated .part file,
// so no merge step is needed. Progress is checkpointed to SQLite every ~1 s.
```

- If the server doesn't support ranges, fall back to a single connection with no resume, and warn the user.
- On resume, compare ETag / Last-Modified /size. If any has changed, restart the download (and ask the user first).
- Each worker has its own retry with exponential backoff and jitter. 429 / 503 responses respect Retry-After.
- If a worker hits repeated connection errors, the job lowers its connection count (adaptive backoff).

4.4 Integrity & finalization

- Optional checksum (MD5 / SHA-1 / SHA-256 / SHA-512) verified after completion.
- Atomic finalize: file.part is fsynced, then renamed to file. The rename-on-conflict policy is applied here.

4.5 Protocol support

PROTOCOL	PHASE	NOTES
HTTP / HTTPS (HTTP/1.1, HTTP/2)	v1	Custom headers, cookies, basic auth, referer, user-agent
Proxies: HTTP, SOCKS5	v1	Global or per job; system proxy detection
FTP / SFTP	v2	REST-based resume

BitTorrent / magnet	v2	anacrolix/torrent
HLS / DASH, site extractors	v3	Native HLS; optional yt-dlp backend

5 Batch downloads

A **batch** is a named group of downloads managed as one unit. Every item in a batch is a normal job linked by `batch_id`, so segmentation, pause/resume and retry work unchanged. The batch only adds grouping, aggregate reporting, ordering and batch-level policies.

5.1 Ways to create a batch

SOURCE	BEHAVIOR	PHASE
Paste list	Multi-line text. Extracts every URL, ignores junk, and removes duplicates.	v1
Import file	<code>.txt</code> (one per line), <code>.csv</code> (url, filename, folder), <code>.json</code> , <code>.metalink</code>	v1
Pattern / range URL	<code>img[001-250].jpg</code> (zero-padding kept), <code>[a-z]</code> , stepped <code>[0-100:5]</code> , several ranges in one URL. Shows a live preview and count.	v1
Page link grabber	Fetches the HTML and extracts <code>a[href]</code> , <code>img</code> , <code>video</code> , <code>source</code> . Filters by extension, regex, domain and size.	v2
Directory listing	"Index of /" and FTP folders with optional recursion, preserving the folder structure	v2
Browser "download all"	The extension sends the selected or all links as one batch	v2
Playlists	Through the <code>yt-dlp</code> backend	v3

5.2 Review dialog

- **Items table** with a checkbox, URL, detected name, size, type icon and probe status. A bounded worker pool (default 8) probes in parallel and fills in rows live.
- **Filters:** extension chips, min/max size, include/exclude regex, and "select all / invert".
- **Rename template** with a live preview. Tokens: `{name}` `{ext}` `{index}` `{index:03}` `{batch}` `{date}` `{host}`.
- **Destination:** a folder, an auto-created subfolder named after the batch, or the remote path structure kept.
- **Batch settings:** order (parallel or sequential), max parallel items, connections per item, speed limit, queue, and start (now, paused, or scheduled).
- **Conflict policy:** skip / overwrite / auto-rename / resume existing `.part`.

5.3 Running a batch

- Appears as one **expandable card** with aggregate progress, combined speed, ETA and counters (*37 / 120 done · 2 failed*).
- **Group actions:** pause, resume, cancel, set priority, change limit, **retry failed only**.
- Per-item control still works inside the expanded view. Items can be reordered by dragging.
- **Failure policy:** retry each item *k* times, then either continue with the others or stop the batch.
- **Per-host politeness:** a cap on total connections to one host across the batch, to avoid bans.
- **Link refresh:** re-import a list and match items by filename to swap in fresh signed URLs while keeping partial data.

5.4 After completion

- One summary notification (*"Season 1 — 12 files, 8.4 GB, 1 failed"*).
- Batch post-actions: extract all, join split archives, verify against a `SHA256SUMS` file, run a script, sleep or shut down.
- Export the failed URLs, or the whole batch, as `.txt` or `.metainfo`. Saved **batch templates** hold reusable settings.

5.5 Concurrency model

Three nested limits: **global active jobs** $\geq$ **batch max parallel items** $\geq$ **connections per job**. On top of these sit per-host caps and three layers of token-bucket bandwidth limits (global $\rightarrow$ batch $\rightarrow$ job). The scheduler starts the highest-priority eligible job whenever a slot frees up.

6 Queues, scheduling & limits

- **Multiple queues**, each with its own concurrency, speed limit and schedule. A default "Main" queue is created on first run.
- **Scheduler**: time windows (e.g. 02:00–07:00), weekday masks, and "stop when the window ends".
- **Profiles**: *Full speed*, *Balanced*, *Metered*. Switchable from the tray or header in one click.
- **Bandwidth limit**: global, per queue, per batch and per job. The limit can be changed live without restarting connections.

7 Capture & integration

FEATURE	DESCRIPTION	PHASE
Add-URL dialog	Paste a URL. Filename, size and folder are prefilled after probing.	v1
Clipboard monitor	Offers to download when a URL is copied (opt-in, with an extension whitelist)	v1
Drag & drop	Drop links, text files or <code>.metalink</code> onto the window	v1
Tray + notifications	Tray menu (pause all, profile, show) and freedesktop notifications	v1
CLI companion	<code>adm add URL...</code> , <code>adm batch file.txt</code> , <code>adm ls</code> , <code>adm pause ID</code>	v2
Browser extension	Chrome/Firefox capture of downloads and "download all links"	v2
MIME / scheme handler	<code>magnet:</code> , <code>.torrent</code> , <code>.metalink</code> through <code>xdg-mime</code>	v2

8 Organization & automation

- **Auto-categories**: Video, Music, Archives, Documents, Images, Programs, Other. Each has a default folder under `~/Downloads`.
- **Rules engine** v2: conditions (domain, extension, size, regex) → actions (folder, queue, connections, post-action).
- **Post-download actions**: open, reveal in folder, extract, run script, mark executable.
- **History** with full-text search, re-download, and duplicate warnings by URL or target path.

9 UI / UX design

Design principles: **calm by default, detailed on demand**. The main list stays readable at a glance. The details drawer shows everything else. Motion is used only to communicate change (progress, reordering, state transitions), and honors *prefers-reduced-motion*.



Figure 2. Main window concept (dark theme). Multi-colored bars show each connection's segment.

Key UI elements

- **Sidebar:** status filters, categories, queues and batches, each with live counts.
- **Header:** a combined search / quick-add box (`Ctrl K`), a global speed sparkline and the profile switcher.
- **Download card:** icon, name, meta line, percent, and a **per-segment progress bar**. Batch cards expand to show their items.
- **Details drawer:** overview, segments map, HTTP headers, event log, checksum, and "copy as curl".
- **Dialogs:** New download, New batch (with a Paste / Import / Pattern tabs), Settings.
- **Empty states** with drop-zone illustrations; **toasts** for non-blocking feedback.
- Multi-select with `Shift / Ctrl` , a right-click context menu, and full keyboard navigation.

Design tokens

TOKEN	DARK	LIGHT	USE
bg	 #0B1020	 #F6F7FB	App background
surface	 #141B31	 #FFFFFF	Cards, dialogs
accent	 #6366F1	 #4F46E5	Primary actions, active progress
accent-2	 #22D3EE	 #0891B2	Batches, speed graph
success / warn / danger	  	  	Completed · Paused · Failed

Typography: **Inter** (UI), falling back to Ubuntu Sans; **JetBrains Mono** for sizes, speeds and URLs (tabular numbers keep changing values from jittering). Radius: 8 / 12 / 16 px. Spacing on a 4 px grid. Motion: 150–250 ms ease-out, with springy reordering for list changes.

10 Data model

```
CREATE TABLE batches (  
  id TEXT PRIMARY KEY, name TEXT, order_mode TEXT, max_parallel INT,  
  on_failure TEXT, rename_tpl TEXT, dest_dir TEXT, created_at INT  
);  
CREATE TABLE jobs (  
  id TEXT PRIMARY KEY, batch_id TEXT REFERENCES batches(id), queue TEXT,  
  url TEXT, final_url TEXT, filename TEXT, dir TEXT, size INT,  
  downloaded INT, status TEXT, error TEXT, etag TEXT, last_modified TEXT,  
  resumable INT, connections INT, speed_limit INT, priority INT, position INT,  
  headers_json TEXT, checksum TEXT, created_at INT, completed_at INT  
);  
CREATE TABLE segments (  
  job_id TEXT REFERENCES jobs(id) ON DELETE CASCADE, idx INT,  
  start INT, "end" INT, done INT, PRIMARY KEY(job_id, idx)  
);
```

11 Non-functional requirements

AREA	TARGET
Throughput	Saturate a 1 Gbit/s link with 16 connections on a range-capable server
UI performance	Progress events are coalesced to ≤ 10 Hz per job. 500 rows scroll smoothly (virtualized list).
Resource use	Idle memory < 80 MB. CPU < 5% while downloading at 100 MB/s.
Durability	<code>kill -9</code> at any moment loses at most ~1 s of progress and never corrupts data
Security	TLS verification on by default. Local API on a user-only socket (0600). No telemetry.
Accessibility	Full keyboard operation, visible focus rings, WCAG AA contrast in both themes
Testing	Engine unit tests against an <code>httptest</code> server that simulates ranges, drops, throttling and 5xx errors

12 Roadmap

MILESTONE	SCOPE
M1 • Engine	Probe, dynamic segmentation, pause/resume, retry, rate limit, SQLite persistence, tests
M2 • Shell & UI	Wails app, sidebar/list/cards, add dialog, per-segment bars, themes, details drawer
M3 • Batch	Paste/import/pattern, review dialog with probing, filters, rename templates, batch cards, retry-failed
M4 • Polish	Queues and scheduler, tray, notifications, clipboard monitor, settings, packaging (ApplImage/.deb)
v2	CLI, browser extension, rules engine, page link grabber, FTP/SFTP, BitTorrent
v3	HLS/DASH, yt-dlp backend, LAN remote UI, plugins

13 Risks & open questions

- **WebKitGTK variance** across distributions: target 4.1, and test on Ubuntu, Fedora and Arch. Flatpak removes most of this variance.
- **Servers that lie** (wrong `Content-Length`, ranges ignored mid-stream): validate each response's `Content-Range`, and fall back to a single connection.
- **Aggressive multi-connection use** may trigger rate limits or bans. Defaults are sensible, and per-host caps are included.
- **Browser extension** packaging and store review take time. Deferred to v2.
- *Open*: the final product name, and whether to support a headless daemon mode for servers in v1.